

学号		成绩	
----	--	----	--



沈阳建筑大学

编译原理上机报告

名 称 编译原理实验四
 学 院 信息与控制工程学院
 专 业 计算机科学与技术
 班 级 计算机 1902 班
 姓 名 蔡艺洵

2021 年 12 月 7 日

一、上机目的

1、目的

通过上机实习，加深对语法制导翻译原理的理解，掌握将语法分析所识别的语法范畴变换为某种中间代码的语义翻译方法。

2、要求

- (1) 选用目前世界上普遍采用的语义分析方法——语法制导翻译技术。
- (2) 语义分析对象重点考虑经过语法分析后已是正确的语法范畴，实习重点是语义子程序。
- (3) 中间代码选用比较常见的形式，例如四元式。

二、基本原理和上机步骤

本次上机实验有两个实验可供选择，为了上机实验的顺利进行，这边需要提前了解相关的知识。

首先需要知道，属性有两类：

综合属性：归约型属性，用于“自下而上”传递信息。

继承属性：推导型属性，用于“自上而下”传递信息。

综合属性的例子：

产生式	语义规则
$L \rightarrow E$	$Print(E.val)$
$E \rightarrow E^1 + T$	$E.val := E^1.val + T.val$
$E \rightarrow T$	$E.val := T.val$
$T \rightarrow T^1 * F$	$T.val := T^1.val \times F.val$
$T \rightarrow F$	$T.val := F.val$
$F \rightarrow (E)$	$F.val := E.val$
$F \rightarrow digit$	$F.val := digit.lexval$

继承属性的例子：

产生式	语义规则
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L^1, id$	$L^1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

语法制导

翻译的基本思想：为每个产生式配上一个语义子程序，（该子程序描述了一个产生式所对应的翻

译工作。这些工作包括：生成中间代码，查填有关的符号表，检查和报错，修改编译程序某些工作变量的值等)。在语法分析过程中，每当一个产生或用于匹配式进行归约时，就调用该产生式所对应的语义子程序，以完成即定的翻译任务。

主要的中间代码有：逆波兰、四元式、三元式、间接三元式、树。

本次上机有两个实验可选择，我选择了实现逆波兰式的实验。

要求：

1、逆波兰式定义

将运算对象写在前面，而把运算符号写在后面。用这种表示法表示的表达式也称做后缀式。逆波兰式的特点在于运算对象顺序不变，运算符号位置反映运算顺序。采用逆波兰式可以很好的表示简单算术表达式，其优点在于易于计算机处理表达式。

2、产生逆波兰式的前提

中缀算术表达式

3、逆波兰式生成的实验设计思想及算法

三、上机结果

程序输入/输出用例：

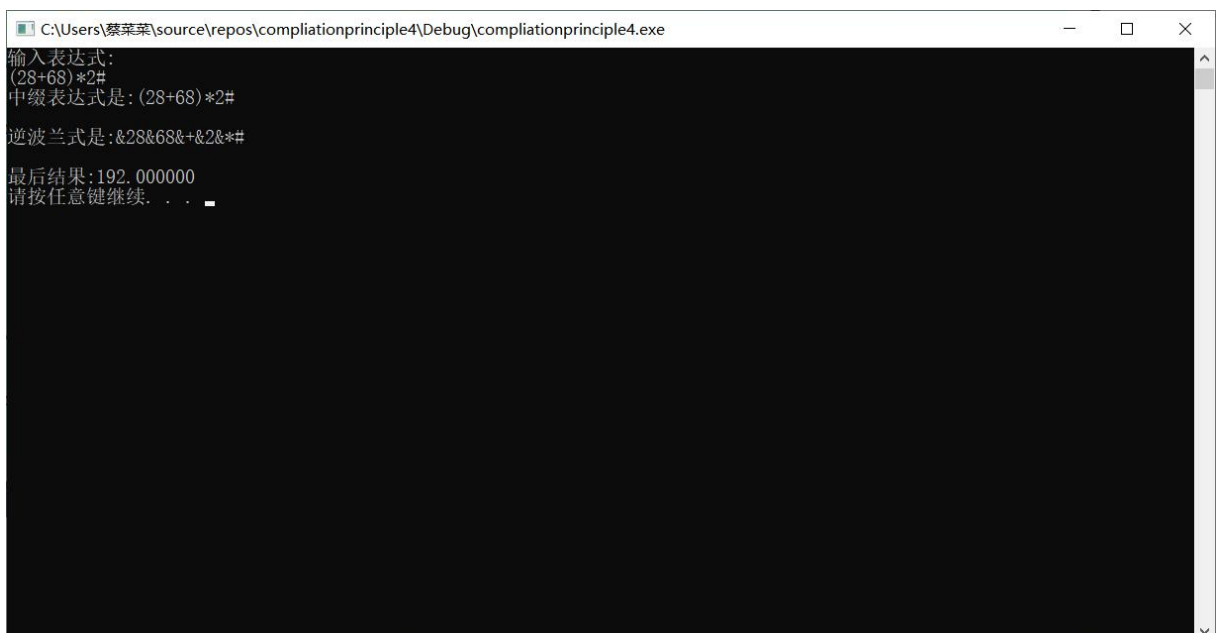
输出的格式如下：

(1)输入一以#结束的中缀表达式(包括+—*/（）数字#)：在此位置输入符号串如(28+68)*2#

(2)逆波兰式为：&28&68&+&2&*#

(3)逆波兰式 28&68+2*计算结果为 192

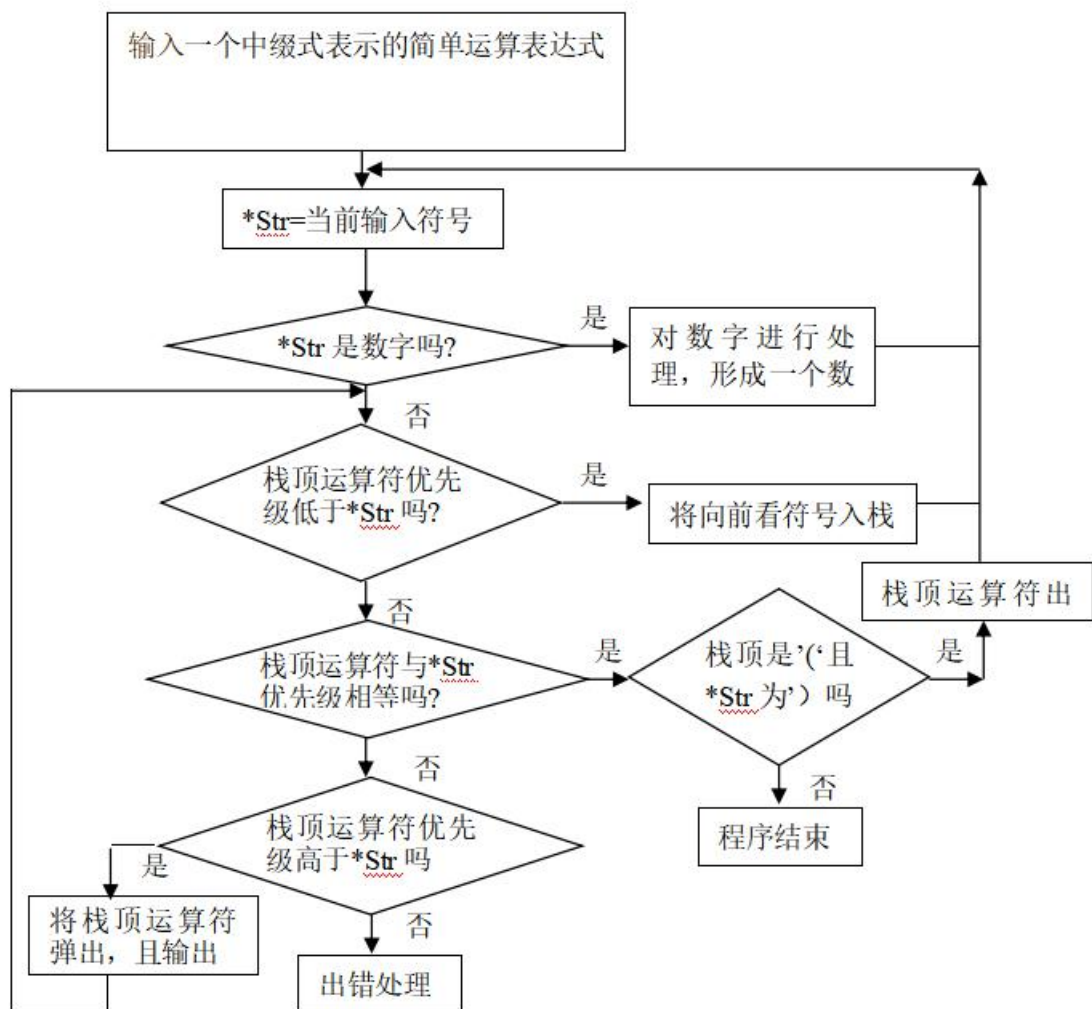
运行结果图：



```
C:\Users\蔡某某\source\repos\compilationprinciple4\Debug\compilationprinciple4.exe
输入表达式:
(28+68)*2#
中缀表达式是: (28+68)*2#
逆波兰式是: &28&68&+&2&*#
最后结果:192.000000
请按任意键继续. . .
```

四、讨论与分析

本次上机实验的业务逻辑图如下所示：



(1)首先构造一个运算符栈，此运算符在栈内遵循越往栈顶优先级越高的原则。

(2)读入一个用中缀表示的简单算术表达式

(3)从左至右扫描该算术表达式，从第一个字符开始判断，如果该字符是数字，则分析到该数字串的开始并将该数字串直接输出。

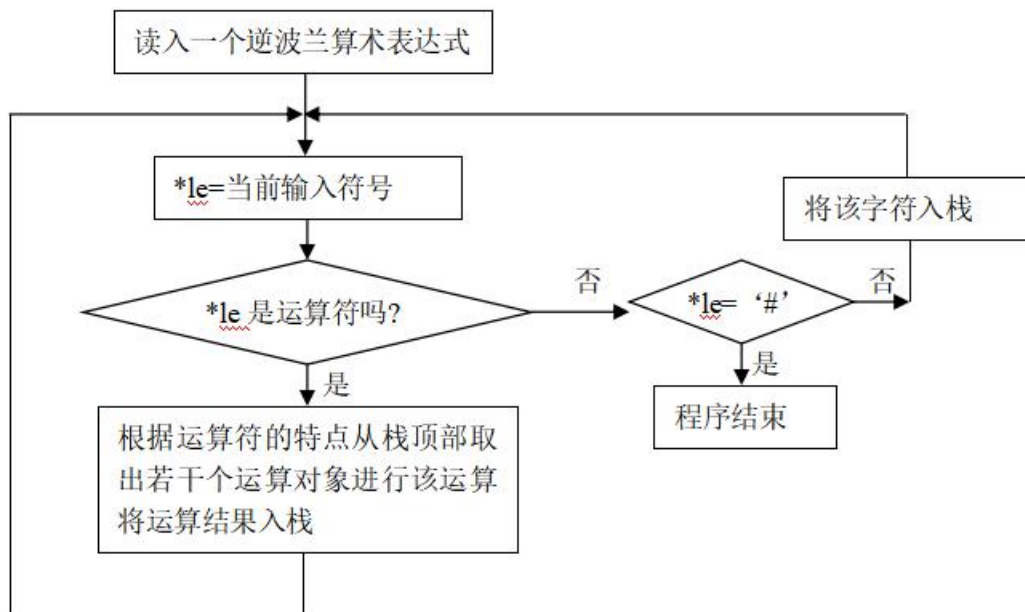
(4)如果不是数字，该字符则是运算符，此时需比较优先关系。

做法如下：将该字符与运算符栈顶的运算符的优先关系相比较。如果，该字符优先关系高于此运算符栈顶的运算符，则将该运算符入栈。倘若不是的话，则将此运算符栈顶的运算符从栈中弹出，将该字符入栈。

(5)重复上述操作(1)-(2)直至扫描完整个简单算术表达式，确定所有字符都得到正确处理，我们

便可以将中缀式表示的简单算术表达式转化为逆波兰表示的简单算术表达式。

至此，我们实现了将中缀表达式转换成逆波兰式，按照实验要求，下一步我们要接种逆波兰式进行计算。该功能借助一个函数来实现，业务逻辑图如下：



(1)构造一个栈，存放运算对象。

(2)读入一个用逆波兰式表示的简单算术表达式。

(3)自左至右扫描该简单算术表达式并判断该字符，如果该字符是运算对象，则将该字符入栈。若是运算符，如果此运算符是二目运算符，则将对栈顶部的两个运算对象进行该运算，将运算结果入栈，并且将执行该运算的两个运算对象从栈顶弹出。如果该字符是一目运算符，则对栈顶部的元素实施该运算，将该栈顶部的元素弹出，将运算结果入栈。

(4)重复上述操作直至扫描完整个简单算术表达式的逆波兰式，确定所有字符都得到正确处理，我们便可以求出该简单算术表达式的值。

对于本次实验，我参考了数据结构的相关知识，也提前在本次实验之前了解了相关的课堂知识，知道了如何构建识别中缀表达式，并将其转换成后缀表达式的过程。充分的理论知识是我实践编写代码的基石，这也说明了，理论知识是动手实践的指导。不管是实现哪种算法、或者是哪种分析法，都是需要我们去了解他理论层面的东西，虽然这和代码实践会有一些在细节上处理的不一样，但是大体框架、流程都是依据理论层面的知识来进行搭建了，只是说，不同的人，可能有不同的搭建方法而已。

五、源程序清单

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "string.h"
#include "stdlib.h"

#define MAXLEN 100
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

typedef struct
{
    char op;
    int level;
}opt;

typedef struct          //定义操作符栈
{
    opt st[MAXLEN];
    int top;
}op_stack;

typedef struct          //定义值栈
{
    double D[MAXLEN];
    int top;
}D_stack;

//-----对栈操作的定义-----
opt peek(op_stack *s)    //定义看栈顶函数
{
    opt error = { '$', -2 };
    if (s->top >= 0)
        return s->st[s->top];
    else
        return error;
}

int IsEmpty(op_stack *s)    //定义判断栈空的函数
```

```

{

    if (s->top < 0)
        return 0;
    else
        return s->st[s->top].op;
}

char push(op_stack *s, opt c)    //定义入栈函数
{
    s->top++;
    s->st[s->top] = c;
    return c.op;
}

opt pop(op_stack *s)    //定义出栈函数
{
    opt i;
    opt error = { '$', -2 };
    if (s->top >= 0)
    {
        i = s->st[s->top];
        s->st[s->top].op = '\0';
        s->top--;
        return i;
    }
    else
        return error;
}

void clear(op_stack *s)    //定义初始化栈
{
    s->top = -1;
}

//-----define the value stack-----
double Dpeek(D_stack *s)    //定义看栈顶函数
{
    if (s->top >= 0)
        return s->D[s->top];
    else
        return 0;
}

```

```

int DIsEmpty(D_stack *s)          //定义判断栈空的函数
{

    if (s->top < 0)
        return 0;
    else
        return (int) (s->D[s->top]);
}

```

```

double Dpush(D_stack *s, double c)    //定义入栈函数
{
    s->top++;
    s->D[s->top] = c;
    return c;
}

```

```

double Dpop(D_stack *s)              //定义出栈函数
{
    double i;
    if (s->top >= 0)
    {
        i = s->D[s->top];
        s->D[s->top] = '\0';
        s->top--;
        return i;
    }
    else return 0;
}

```

```

void Dclear(D_stack *s)              //定义初始化栈
{
    s->top = -1;
}

```

//-----定义栈的操作完成-----

//-----中缀变逆波兰式的函数-----

```

void change(char *str, char *expr, char *kkk)
{
    opt A = { '+', 1 };
    opt R = { '-', 1 };
    opt M = { '*', 2 };
    opt D = { '/', 2 };
}

```

```

opt B = { '(', -1 };
opt Mo = { '%', 2 };

op_stack os;
clear(&os);
while (*str != '#')
{

    while (*str >= '0' && *str <= '9' || *str == '.')
    {
        *expr = *str;
        expr++;
        str++;
    }

    *expr++ = '&';


    switch (*str)
    {
    case '+':
        while (peek(&os).level >= A.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;
        }
        push(&os, A);
        str++;
        break;
    case '-':
        while (peek(&os).level >= R.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;
        }
        push(&os, R);
        str++;
        break;
    case '*':
        while (peek(&os).level >= M.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;

```

```

    }
    push(&os, M);
    str++;
    break;
case '/':
    while (peek(&os).level >= D.level && IsEmpty(&os))
    {
        *expr++ = pop(&os).op;

    }
    push(&os, D);
    str++;
    break;
case '(':
    push(&os, B);
    str++;
    break;
case ')':
    while (peek(&os).level != -1)
    {
        *expr++ = pop(&os).op;
    }
    pop(&os);
    str++;
    break;
case '#':
    while (IsEmpty(&os))
    {
        *expr++ = pop(&os).op;
        *expr++ = '&';//must add this line
    }
    *expr--;
    *expr = '#';
    break;
case '%':
    while (peek(&os).level >= Mo.level)
    {
        *expr++ = pop(&os).op;
    }
    push(&os, Mo);
    str++;
    break;
}
}

```

```
}
```

```
//-----后缀求值函数-----
```

```
double calval(char *le)
{
    int i, leng;
    double result = 0;
    D_stack Data;
    double tmpData, tmp1;
    char tmp[MAXLEN] = { '\0' };
    int a = 0;
    Dclear(&Data);
    while (*le != '\0')
    {
        while (*le >= '0' && *le <= '9' || *le == '.')
        {
            tmp[a++] = *le++;
        }
        le++;
        tmp[a] = '\0';
        tmp1 = atof(tmp);
        if (tmp1 != 0.0) {
            Dpush(&Data, tmp1);
            leng = strlen(tmp);
            for (i = 0; i < leng; i++) //重新初始化临时数组
                tmp[i] = '\0';
            a = 0;
        }
        switch (*le)
        {
            case '&':
                le++;
                break;
            case '#':
                le++;
                break;
            case '+':
                result = Dpop(&Data) + Dpop(&Data);
                Dpush(&Data, result);
                le++;
                break;
            case '-':
                result = Dpop(&Data) - Dpop(&Data);
                Dpush(&Data, result);
```

```

        le++;
        break;
    case '*':
        result = Dpop(&Data)*Dpop(&Data);
        Dpush(&Data, result);
        le++;
        break;
    case '/':
        result = Dpop(&Data) / Dpop(&Data);
        Dpush(&Data, result);
        le++;
        break;
    case '%':
        tmpData = Dpop(&Data);
        result = (double)((int)Dpop(&Data) % (int)tmpData);
        Dpush(&Data, result);
        le++;
        break;
    }
}

return result;

}

/*
int BracketCorrespondency(char *str)
{
    int i = 0;
    op_stack s;
    clear(&s);
// Stack s;
// InitStack(s);
    char x;
    while (str[i]){
        switch (str[i]){
            case '(':
                Push(s, str[i]);
                break;
            case '[':
                Push(s, str[i]);
                break;
            case ')':
                GetTop(s, x);
                if (x == '(') Pop(s, x);

```

```

        else return FALSE;
        break;
    case ']' :
        GetTop(s, x);
        if (x == '[') Pop(s, x);
        else return FALSE;
        break;
    default:
        break;
    }
    i++;
}
if (s.size != 0) return FALSE;
return TRUE;
}
*/

//-----主函数-----

void main()
{
    char string[MAXLEN];
    char exp[MAXLEN] = { '\0' };
    //char exp[MAXLEN] = "1#2#*3#4#*#+";
    //-----数据定义结束-----
    char *kkk;
    kkk = exp;
    puts("输入表达式:");
    gets_s(string);
    printf("中缀表达式是:%s\n\n", string);
    change(string, exp, kkk);

    printf("逆波兰式是:%s\n", exp);

    printf("\n 最后结果:%f\n", calval(exp));

    system("PAUSE");

}

```